
SAVeKART

Mathematical Calculations & Formulations

Hubli-Dharwad, Karnataka — Tier-2 City

Mathematical Models, Optimization Functions & Statistical Computations

Linear Programming · TSP · Haversine Distance

Exponential Decay · Bin-Packing · Sensitivity Analysis

Maxima/Minima · Greedy Heuristics · Demand Statistics

Generated 20 September 2026

Table of Contents

1. Haversine Distance Formula
2. Frequency Classification & Demand Statistics
3. Optimization Model — Linear Program
 - 3.1 Objective Function
 - 3.2 Constraints
 - 3.3 On-Time Probability Function
 - 3.4 Warehouse Scoring (Greedy Heuristic)
 - 3.5 Total Cost Components
4. Co-Delivery Optimization
5. Route Optimization — Nearest-Neighbour TSP
6. Order Classification — Maxima/Minima
7. Warehouse Space Optimization — Bin-Packing
8. Reliability Matrix
9. Sensitivity Analysis
10. Summary of Mathematical Techniques

1. Haversine Distance (Great-Circle Distance)

The distance between two points on Earth's surface given their latitude and longitude:

```
d = 2R · arctan(√a / √(1-a))

where:
a = sin²(Δlat/2) + cos(lat₁)·cos(lat₂)·sin²(Δlon/2)
Δlat = lat₂ - lat₁
Δlon = lon₂ - lon₁
R = 6371 km (Earth's radius)
```

Used in: All distance computations between neighbourhoods and warehouses.

Example: Distance from Old Hubli (15.3647, 75.1240) to Gokul Road Logistics Hub (15.3700, 75.1620):

```
Δlat = 0.0053°, Δlon = 0.0380°
a = sin²(0.00265) + cos(15.3647°)·cos(15.3700°)·sin²(0.019°)
a ≈ 0.000007 + 0.9650·0.9650·0.000110 ≈ 0.000113
d ≈ 2 × 6371 × arctan(√0.000113 / √0.999887) ≈ 4.28 km
```

2. Frequency Classification & Demand Statistics

Neighbourhoods are classified into three zones based on daily order volume:

Class	Condition	Color	Action
Red	daily_orders ≥ 200	●	High priority — nearest warehouse
Amber	120 ≤ daily_orders < 200	●	Medium priority
Green	daily_orders < 120	●	Low priority

Statistical measures computed:

```
Mean (μ) = Σ(daily_orders_i) / N
Variance (σ²) = Σ(daily_orders_i - μ)² / N
Std Dev (σ) = √(σ²)
Peak Day Demand = Total Orders × 1.35
```

The **1.35 multiplier** represents the empirical peak-day surge factor observed in Indian Tier-2 city e-commerce delivery patterns (festive seasons, weekends, payday spikes).

3. Optimization Model — Linear Program

3.1 Objective Function

The goal is to **minimize** the total order-weighted delivery cost plus late-delivery penalty:

```
Minimize Z = Σ_i Σ_r x_{ir} · [d_{ir} · c_fuel + n_i · p_late · (1 - P_{ir})]
```

Where:

- x_{ir} = 1 if neighbourhood i is assigned to warehouse j , 0 otherwise
- d_{ir} = Haversine distance from neighbourhood i to warehouse j (km)
- c_{fuel} = Fuel cost per km (₹/km, default ₹8.5)
- n_i = Daily orders from neighbourhood i

- p_{late} = Late delivery penalty per order (₹, default ₹50)
- P_{ir} = Probability of on-time delivery from warehouse j to neighbourhood i

3.2 Constraints

1. $d_{ir} \leq R_{service}$ (service radius, default 5 km)
2. $\sum_r x_{ir} = 1 \forall i$ (each neighbourhood assigned to exactly one warehouse)
3. $\sum_r y_r \leq K_{max}$ (maximum number of warehouses, default 5)
4. $x_{ir} \leq y_r \forall i, j$ (neighbourhood can only be assigned to a selected warehouse)
5. $x_{ir} \in \{0, 1\}$ (binary assignment)
6. $y_r \in \{0, 1\}$ (binary warehouse selection)

3.3 On-Time Probability Function

The probability that an order arrives on time decreases exponentially with distance:

$$P_{ir} = r_i \cdot e^{(-d_{ir} / R)}$$

where:

r_i = base on-time rate for neighbourhood i (historical)
 d_{ir} = distance from warehouse j to neighbourhood i
 R = service radius (km)

Interpretation: At distance 0, $P = r_i$ (base rate). At distance = R , $P = r_i/e \approx 0.368 \cdot r_i$. This captures the intuition that longer delivery distances reduce reliability.

3.4 Warehouse Scoring Function (Greedy Heuristic)

Since the full integer program is NP-hard, SAVeKART uses a greedy facility-location heuristic:

Step 1: Score each warehouse:

$$\text{Score}(j) = \sum_i [n_i / \max(0.1, d_{ir})] \text{ for all } i \text{ where } d_{ir} \leq R$$

Higher score = warehouse is close to many high-order neighbourhoods.

Step 2: Select top-K warehouses by score.

Step 3: Assign each neighbourhood to the nearest selected warehouse within radius.

Step 4: Compute costs for each assignment.

3.5 Total Cost Components

$$\text{Total Weighted Cost} = \sum_i [d_{ir^*} \cdot c_{fuel} \cdot n_i + n_i \cdot p_{late} \cdot (1 - P_{ir^*})]$$

where j^* = assigned warehouse for neighbourhood i

$$\text{Fuel Cost} = \sum_i d_{ir^*} \cdot c_{fuel} \cdot n_i$$

$$\text{Late Penalty} = \sum_i n_i \cdot p_{late} \cdot (1 - P_{ir^*})$$

$$\text{Coverage (\%)} = (\text{assigned neighbourhoods} / \text{total neighbourhoods}) \times 100$$

4. Co-Delivery Optimization

4.1 Co-Delivery Detection

Two neighbourhoods assigned to the same warehouse are flagged for co-delivery if they are within 0.5 km of each other (same building or adjacent houses):

$$\text{Co-delivery}(i, j) = \text{true if } d(i, j) \leq 0.5 \text{ km AND } \text{warehouse}(i) = \text{warehouse}(j)$$

4.2 Savings Calculation

When co-delivery is detected, the fuel saved from combining trips is:

$$\text{Fuel Saved} = d(i, j) \times c_{\text{fuel}} \times 2$$
$$\text{Savings per Order} = (\text{Fuel Saved} / (n_i + n_r)) \times (\text{discount\%} / 100)$$
$$\text{Total Savings} = \sum \text{Co-delivery pairs } [\text{savings_per_order} \times (n_i + n_r)]$$

The **discount percentage** (default 8%) represents the portion of fuel savings passed to the customer as a benefit for opting into the optimized co-delivery route.

5. Route Optimization — Nearest-Neighbour TSP

5.1 Algorithm

For each warehouse, the delivery route is optimized using a nearest-neighbour heuristic for the Traveling Salesman Problem (TSP):

1. Start at warehouse position (lat_w, lon_w)
2. Find the unvisited stop nearest to current position
3. Move to that stop, mark as visited
4. Repeat until all stops visited
5. Return to warehouse

$$\text{Route distance} = \sum d(\text{stop}_k, \text{stop}_{\{k+1\}}) + d(\text{last_stop}, \text{warehouse})$$

5.2 Time Window Estimation

$$\text{Total Time (min)} = (\text{Total Distance} / \text{Avg Speed}) \times 60 + (\text{Num Stops} \times 5 \text{ min})$$

where Avg Speed = 25 km/h (urban Hubli-Dharwad traffic)

5 min = average time per delivery stop

5.3 Fuel Cost per Route

$$\text{Route Fuel Cost} = \text{Total Distance} \times c_{\text{fuel}}$$

6. Order Classification — Maxima/Minima Application

6.1 Classification Function

Orders are classified based on weight and volume using threshold boundaries (maxima/minima of a weighted scoring function):

Fragility Score = f(weight, volume) = weight × 0.3 + volume × 0.7

Classification:
Fragile if declared_fragile = true OR weight < 2 kg
Sturdy if weight > 15 kg OR volume > 3 cft
Normal otherwise

- Maxima/Minima interpretation:**
- **Minima:** Weight < 2 kg → minimum weight threshold for fragile classification (light items are likely delicate)
 - **Maxima:** Weight > 15 kg or volume > 3 cft → maximum threshold for sturdy classification (heavy/bulky items need robust transport)

6.2 Vehicle Selection Function

The cost-effectiveness function for vehicle selection:

Cost-Effectiveness = (Load Capacity) / (Cost per km × Fuel Efficiency)

Vehicle	Type	Capacity	Cost/km	Efficiency
Electric Van (Bajaj RE EV)	Fragile	200 kg	₹2.5	EV (zero fuel)
Piaggio Ape	Normal	500 kg	₹4.0	25 km/L
Tata Ace	Sturdy	1000 kg	₹6.0	15 km/L

The vehicle with the best cost-effectiveness ratio for the given order type is recommended, minimizing:

Transport Cost = (Distance × Cost per km) + (Damage Risk × Repair Cost)

7. Warehouse Space Optimization — Bin-Packing

7.1 Space Allocation

Capacity (cft) = floor_area (sq ft) × ceiling_height (8 ft)
Used Space = allocated_orders × avg_order_volume (cft)
Utilization (%) = (Used Space / Capacity) × 100
Overflow = ceil((Used Space - Capacity) / avg_order_volume) if > 100%

7.2 Load Utilization

Load Utilization (%) = min(100, (total_orders × 2 sq ft/order) / capacity_sqft × 100)

The **2 sq ft per order** is an empirical estimate for standard parcel storage in Indian warehousing.

8. Reliability Matrix

The full reliability matrix computes on-time probability for every neighbourhood-warehouse pair within the service radius:

```
Matrix[i][j] = {{
  distance: dir,
  on_time_prob: ri × e-(dir/R)
}} for all dir ≤ R
```

This matrix informs the optimization model's late-penalty term and provides visibility into which neighbourhoods have poor delivery reliability.

9. Sensitivity Analysis

SAVeKART runs the optimization model across varying parameters to understand system behavior:

9.1 Service Radius Sensitivity

```
R ∈ {3, 4, 5, 6, 7} km
Observes: total cost, coverage %, warehouses used
```

Expected behavior: Larger radius → fewer warehouses needed, higher coverage, but potentially higher per-delivery cost.

9.2 Late Penalty Sensitivity

```
p_late ∈ {₹0, ₹25, ₹50, ₹75, ₹100}
Observes: total cost, coverage %, warehouses used
```

Expected behavior: Higher penalty → optimizer prefers closer warehouses even if it means selecting more warehouses.

9.3 Warehouse Count Sensitivity

```
K_max ∈ {2, 3, 4, 5, 6}
Observes: total cost, coverage %, warehouses used
```

Expected behavior: More warehouses allowed → lower total cost (closer assignments), but higher fixed costs.

10. Summary of Mathematical Techniques Used

Technique	Application
Haversine formula	Distance calculation between GPS coordinates
Linear programming	Warehouse location optimization (objective + constraints)
Greedy heuristic	Facility-location problem (NP-hard approximation)
Exponential decay	On-time probability as function of distance
Nearest-neighbour TSP	Route optimization for delivery partners
Bin-packing	Warehouse space allocation
Statistical measures	Demand analysis (mean, std dev, variance, peak estimation)
Maxima/minima thresholds	Order classification (fragile/normal/sturdy)
Cost-effectiveness ratio	Vehicle selection optimization
Sensitivity analysis	Parameter tuning and system behavior analysis

References

1. Daskin, M.S. (2013). Network and Discrete Location: Models, Algorithms, and Applications. Wiley.
2. Applegate, D.L. et al. (2006). The Traveling Salesman Problem: A Computational Study. Princeton University Press.
3. Croes, T. (1958). "A Method for Solving Traveling-Salesman Problems." Operations Research, 6(6), 791-812.
4. Census of India 2011 — Hubli-Dharwad demographic data.
5. Indian Road Congress (IRC) standards for urban traffic speed estimation.